

Lab 1.3 — Mapeo de Superficie de Ataque

Qué es este lab

Es una herramienta de auditoría, no un ataque: inspecciona un agente ADK ya construido (sin ejecutarlo) y produce un reporte estructurado de su superficie de ataque, clasificada en las 5 categorías del framework del capítulo (canales de input, memory stores, interfaces de tools, canales inter-agente, datos de entrenamiento). El punto pedagógico es que un equipo de seguridad pueda mapear el riesgo de un agente ANTES de que alguien lo ataque, en vez de descubrirlo después de un incidente.

Las piezas que corren

- **map_attack_surface(agent)** : la función de auditoría. Usa `inspect.signature` sobre cada tool del agente para listar sus parámetros, clasifica cada tool como **HIGH** risk si su nombre contiene palabras como `credit` / `transfer` / `delete` / `admin` (y **MEDIUM** si no), detecta el modelo configurado, y arma un score numérico ponderado por riesgo: cada tool suma 5 puntos si es **HIGH** o 2 si es **MEDIUM**, más `input_channels*2 + memory_stores*1.5`.
- **_extract_model_name(agent)** : helper que devuelve el nombre del modelo aunque `agent.model` sea un objeto `LiteLlm` en vez de un string plano (que es lo que usan todos los agentes de este seminario adaptados a Ollama).
- **benign_agent** y **high_risk_agent** : dos agentes de demostración, contruidos solo para tener algo real que auditar. Reproducen el perfil de Lab 1.1 (tools de filesystem, riesgo **MEDIUM**) y de Lab 1.2 (tools de crédito + una tool administrativa extra, riesgo **HIGH**). Ambos se instancian con `LiteLlm(model="ollama_chat/qwen3.5:9b", ...)` -pero **nunca se invocan**: este script no llama a ningún LLM en ningún momento.

El flujo

1. Se instancian **benign_agent** (tools `read_file` / `write_file`) y **high_risk_agent** (tools `issue_credit` / `delete_user_account`).
2. Para cada uno, **map_attack_surface()** recorre `agent.tools` con `inspect` y arma la lista `tool_interfaces` con nombre, parámetros y nivel de riesgo.

3. Se extrae el nombre del modelo (vía `_extract_model_name`) para el canal de input `text`, y se decide si agregar el canal `image` según si el nombre del modelo sugiere multimodalidad (`flash` / `pro`).
4. Se chequea si el agente tiene memoria de largo plazo configurada (`agent.memory`) o solo memoria de sesión.
5. Se calcula el `attack_surface_score` y se imprime el reporte completo en JSON.
6. `_verify_reports()` corre 7 asserts (dos de ellos dentro de un loop que itera sobre ambos agentes, 9 verificaciones en total) sobre ambos reportes: confirma clasificación de riesgo por tool, que `high_risk_agent` tenga mayor score que `benign_agent` (ver más abajo), y que el nombre del modelo se extrajo correctamente del wrapper `LiteLlm`.

Por qué importa la verificación

Este lab no usa ningún LLM, así que no hay "evidencia de efecto" de tool calls que confirmar -pero sí hay algo más básico que verificar: que la lógica de clasificación e introspección sea correcta, no solo que "no explote". Los asserts de `_verify_reports()` cumplen ese rol.

Un punto de diseño que vale la pena pensar antes de mirar el resultado: si `attack_surface_score()` contara *cantidad* de tools, canales y memory stores sin ponderar por el campo `risk` que la misma función ya calculó, dos agentes con la misma cantidad de tools terminarían con el mismo score aunque tengan perfiles de riesgo completamente distintos -por ejemplo, uno que solo lee y escribe archivos (`MEDIUM` / `MEDIUM`) contra uno que emite créditos y borra cuentas (`HIGH` / `HIGH`). Este lab pondera cada tool según su propio nivel de riesgo (`HIGH` pesa más que `MEDIUM`) para evitar exactamente ese problema.

Resultado esperado

Dos reportes JSON completos (uno por agente), cada uno con `tool_interfaces` clasificadas correctamente, `input_channels` mostrando `"model": "ollama_chat/qwen3.5:9b"` (sin canal `image` agregado porque el nombre no contiene "flash" ni "pro"), y un `attack_surface_score` de 7.5 para `benign_agent` y 13.5 para `high_risk_agent` -la diferencia refleja que `high_risk_agent` tiene tools de mayor riesgo, no solo distinta cantidad. La línea final `[verify] OK: clasificación de riesgo, canal de input y scoring se comportan como se espera.` confirma que los 7 asserts pasaron, incluido el que exige que el score de `high_risk_agent` sea mayor que el de `benign_agent`.